Technical writing test – Siemens (XPE 2026)

Overview

This test comprises two sections, Scenario 1 and Scenario 2. Please attempt both scenarios following the provided Style guide.

Format

Use any authoring tool and submit your work as a PDF file. For bonus points, also include a Markdown (.md) file of your document.

Style guide

Apply these rules consistently throughout your documentation:

- Use active voice and present tense.
- Do not use “we” or “us” to refer to a corporate point of view.
- Avoid the word “user.”
- Do not use contractions.
- Italicize all filenames and pathnames, including the file extension.
- Be consistent with word use and style.
- Use boldface for menu options and buttons (for example, **File** > **Save**).
- Use code formatting for commands and code elements (for example, `cdp policy validate`).
- Use sentence case for headings and titles.

Scenario 1

You are a technical writer at a leading cloud infrastructure company. The engineering team has just released a critical, complex enhancement to your flagship product, Cloud Deployer Pro.

Objective

Lena, a developer on your team, has sent you an email. Using this as your input, develop task-oriented documentation for this feature in accordance with the Style guide.

Include a "Questions for Lena" section at the end of your documentation submission, listing any questions you need answered.

Lena's email

Subject: Cloud deployer pro updated - need new documentation

Hi tech writer, we're nearly ready with the new feature. Info below ...

So the new policy engine for CDP Advanced Multi-Cloud Orchestraton with Policy enforcement is finally out. It's a gamechanger for enteprise. They can now ennforce all their security and compliance rules direct in their deploiment pipelines, across all their clouds, even on-prem stuff we connect too. This isn't just about "if this, then that" checks; they're about declarative state and continuous enforcment.

Okay, so the core concept is a policy-manifest.yaml. This isn't part of the pipeline.yml direct, but it's refferenced. It defines the desired state for resource and configurations. We're using OPA under the hood, but abstracted. Users will need to write policies in a simplified YAML DSL compiles down to Rego. Them define constrains and violaions. A constraint might be "all S3 buckets must be encrypted with KMS keys from this specific ARN," or "no public ingress to databases." A violation is what happens if a constraint isn't met – ether deny the deployment, warn and log, or auto-remediate (which is still experamental for some resource types, be careful there).

The pipeline.yml now has a new top-level policies block. Here, you will have to specifies which policy-manifest.yaml to apply, and crucially, at which stages. You can have different policies for dev, staging, and production. For example, dev might just warn on some violaions, while prod will deny outrite. Also, you can overide specific policy paramaters per stage. This is important for like things environment specific tag requirments or resource size limits.

Credential managament is getting very very tricky. For policy enforcment to work across clouds, our policy engine needs read-only access to all resources that the pipeline might touch, even before they're deploied, to evaluate the desired state against existing

state. This means more granuler IAM roles/service accounts need to be configured for the policy engine itself, seperate from the deploiment roles. We have a new 'Policy Engine IAM Role' section in 'Settings > Cloud Integrations', but it's complex because it needs to cover multiple accounts and potentially different cloud providers for a single policy evaluaton. Cross-account/cross-cloud role assumption is key here.

We've also introduce drift detecton. After a successful deploiment, the policy engine continously monitors the deploied resources against the policy-manifest.yaml and the pipeline.yml's declared state. If something changes outside the pipeline (e.g., someone manually opens an S3 bucket to public), it flags it as drift. Users gets alerts (configurable via webhooks or SNS/SQS integrations). The cdp cli has a new cdp policy detect-drift --pipeline <id> command how they resolve drift is usually manual or by re-running the pipeline, but we're looking into automated remediaton for common cases.

There's a new concept of policy sets. Instead of one giant policy-manifest.yaml, enteprise can define modular policy files (e.g., security-policies.yaml, compliance-policies.yaml, cost-policies.yaml) and compose them into a policy-set.yaml. This policy-set.yaml is then refferenced in the pipeline.yml. This helps with reusability and manage complexity.

The UI for this is still kind of evolving, but the CLI is pretty solid. They'll mostly be interacting with cdp policy validate, cdp pipeline run --policy-check, and cdp policy enforc. Error messages from policy violaions are verbose, but they will need to know how to interpret them, especialy the Rego trace if they're debuging a custom policy.

And don't forget about resource tagging enforcment. Policies can mandate specific tags (e.g., Owner, CostCenter, Environment) on all deploied resources. If a blueprint tries to deploie something without the required tags, the policy should block it. This is a big big one for cost managament and accontability.

Also, we support conditonal policy appliation. A policy might only apply if the target environment is production AND the appliation type is database. This is done with conditions blocks in the policy-manifest.yaml.

This is for the pros, so assume he knows what they're doing with YAML and cloud APIs. Just give them the specifics of our implimentation. Make sure to explain the lifecycle: policy definiton -> policy integraton -> policy evaluaton -> enforcment -> drift detection. And emphasize the immutability principle – policies should ideally to prevent changes outside the pipeline.

Scenario 2

You are the lead technical writer for a new internal tool called "Project Sentinel." This tool helps developers monitor their microservices for performance issues and security vulnerabilities. It has the following main components:

- **Dashboard**: Overview of all monitored services.
- **Service Configuration**: Where developers define which services to monitor and set thresholds.
- **Alerting Rules**: How to set up notifications (email, Teams, PagerDuty) for specific events.
- **Integrations**: How to connect Sentinel with other internal tools (for example, Jira for incident tracking, internal logging system).
- **API Reference**: For developers who want to integrate their custom scripts with Sentinel.

Answer the following questions:

- **Documentation Structure**:
  - Outline the main sections and sub-sections you would propose for the "Project Sentinel" documentation.
  - Explain your organizational choices.
- **User Workflow**:
  - Choose one key user workflow (for example, "Setting up monitoring for a new microservice" or "Configuring an alert for high CPU usage").
  - Describe the steps a user would take and how you would structure the documentation for that specific workflow to make it as easy to follow as possible.
- **Anticipating Challenges**:
  - Identify two potential challenges a developer might face when using Project Sentinel.
  - For each challenge, describe how you would address it in the documentation.
  - Explain why you picked these challenges.